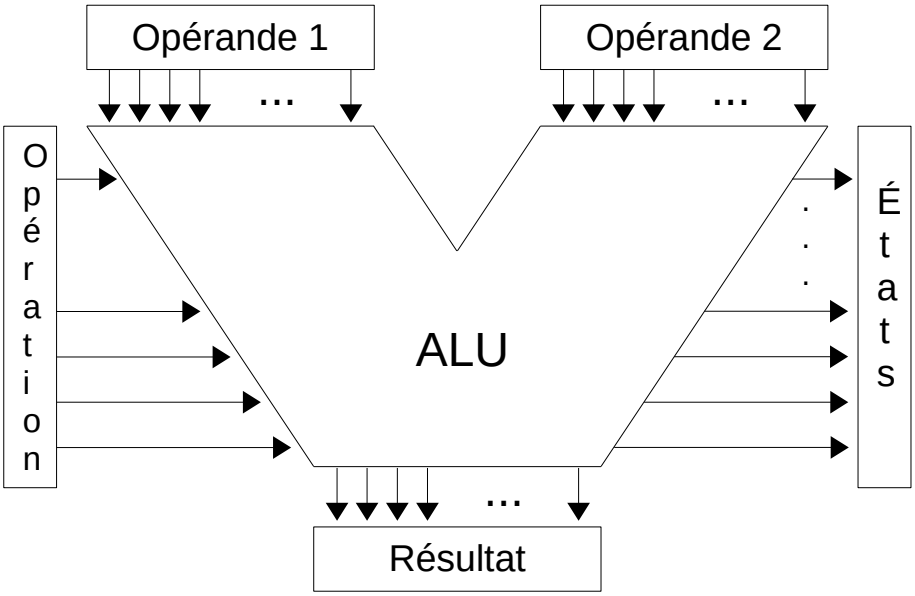
# Un petit bout d'ALU

Pièce maîtresse dans le processeur, l'ALU (Arithmetic and Logical Unit) est le circuit qui effectue les calculs. Historiquement ses capacités de calcul ont beaucoup évolué, partant de quelques opérations sur des entiers courts elles sont capables maintenant d'opérations complexes. On trouve des ALU spécialisées dans les calculs sur des nombres flottants, on parlera de FPU (Floating Point UNIT) ou dans le calcul vectoriel.
L'évolution des processeurs a aussi amené à multiplier les ALU, permettant un traitement parallèle des opérations.

## 1. Structure générique d'une ALU




Représentation schématique classique d'une ALU.

Une ALU peut être représentée schématiquement comme un circuit possédant … des entrées et des sorties, de nature différente :

- 2 groupes d'entrées représentant les opérandes du calcul,
- 1 groupe d'entrée représentant les commandes de l'ALU,
- 1 groupe de sortie représentant la valeur résultat du calcul,
- 1 groupe de sortie représentant l'état du calcul.

1 Entrée : les opérandes

Les opérandes sont les valeurs entrant dans le calcul. Ces valeurs sont présentées dans le code choisie pour la représentation des valeurs, par exemple, cas de loin le plus fréquent, des valeurs entières codées en complément à 2. Ces valeurs seront donc écrites sur plusieurs bits, que l'on peut assimiler à la "puissance" de l'ALU. Ainsi si les valeurs sont sur 8 bits on parlera d'une ALU 8 bits et par extension d'un processeur 8 bits. Pareil pour 16 ou 32 bits.
On peut imaginer que l'ALU est capable de recevoir des valeurs codées sur plusieurs codes différents, mais dans ce cas là il faudra lui "expliquer".
Par exemple, pour augmenter la capacité de l'ALU sans augmenter le nombre de bits des données, on peut choisir de lui fournir des valeurs en binaire naturel, ce qui exclura les valeurs négatives mais doublera la valeur maximale acceptable.
Dans certains cas, un seul opérande peut être utilisé par l'ALU dans l'opération qu'elle réalise.

2 Entrée : les commandes

Les commandes servent à choisir le calcul que l'on désire que l'ALU réalise. Il s'agit d'une série d'entrées, chacune correspondant à une opération, qui devra être mise à 1 pour que l'ALU la réalise, les autres, bien sûr, restant à 0.
On peut trouver comme opérations :

- des opérations arithmétiques : addition, soustraction, multiplication, division, modulo (reste de la division)...
- des opérations logiques : égalité, différence, inférieur, supérieur, inférieur ou égal, supérieur ou égal...
- des opérations sur les bits : décalage à gauche, décalage à droite, ET, OU, XOR, NON...

### 3 Sortie : le résultat

Le résultat est exactement le résultat du calcul sélectionné par les commandes et réalisé par l'ALU.
Il peut faire la même taille que les opérandes, ce qui diminuera la capacité de l'ALU : par exemple l'addition de 2 nombres de N bits tient N+1 bits, ce qui diminue la capacité des opérandes : une ALU de 8 bits ne calculera des additions que sur des nombres de 7 bits. Dans certains cas on peut maintenir la capacité de l'ALU en jouant sur les sorties d'état.
Le résultat peut aussi être prévu plus long que les opérandes, ce qui peut être pratique par exemple dans le cas de la multiplication où le résultat est 2 fois plus long que les opérandes.

### 4 Sortie : l'état du calcul

Ces sorties informent sur le calcul réalisé par l'ALU. Comme pour les commandes, ces informations sont présentées chacune sur un bit qui possède sa propre signification. Certains bits informeront du bon déroulement de l'opération, et d'autres ajouteront des précisions sur le calcul. En anglais ces bits particuliers sont appelés Flag (drapeaux).

## 2. Opérations réalisables par une ALU

L'unité arithmétique et logique est donc le circuit qui réalise les opérations arithmétiques : addition, soustraction, etc. et logiques : égalité, différence, etc.

### 1 Addition – Rappel

L'addition de 2 chiffres binaires est réalisée en calculant les 2 fonctions somme S et retenue R :

| $a$ | $b$ | $S$ | $R$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Avec les fonctions :

$$S = \bar{a}.b + a.\bar{b} = a \oplus b$$

et

$$R_0 = a_0.b_0$$

Pour additionner 2 nombres binaires il suffit de cascader des additions en propageant les retenues : il suffit donc de savoir additionner deux chiffres et une retenue :

| $a_i$ | $b_i$ | $R_{i-1}$ | $S_i$ | $R_i$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

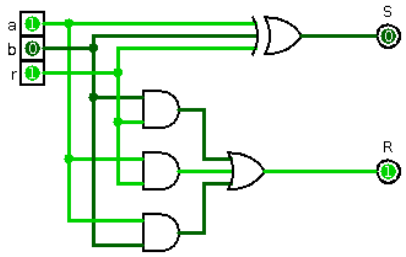
On déduit les fonctions :

$$S_i = \bar{a}.\bar{b}.R_{i-1} + \bar{a}.b.\overline{R_{i-1}} + a.\bar{b}.\overline{R_{i-1}} + a.b.R_{i-1} = a_i \oplus b_i \oplus R_{i-1}$$

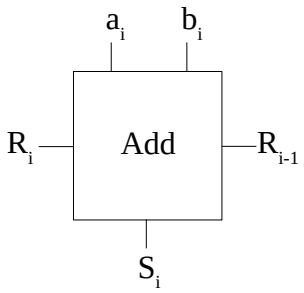
et

$$R_i = a_i.b_i + a_i.R_{i-1} + b_i.R_{i-1}$$

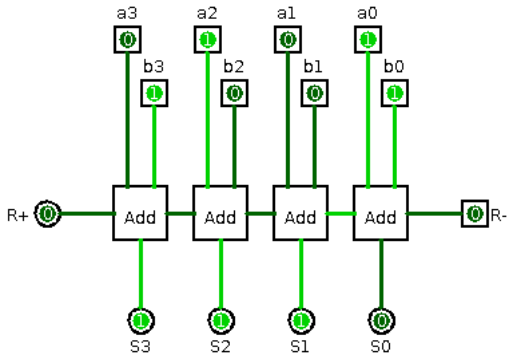
On peut donc construire le circuit élémentaire Additionneur :




Que l'on représentera :

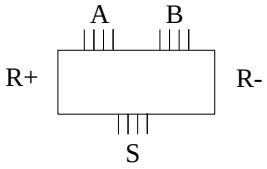



Partant de ce circuit de base le circuit d'addition de deux nombres de longueur quelconque se construit en reproduisant l'algorithme de propagation de la retenue et en utilisant autant d'additionneur élémentaire qu'il y a de chiffres à additionner. Par exemple pour des nombres de 4 bits :




Ce schéma illustre l'addition 5 + 9 = 14

Bien sûr pour des raisons pratiques, ce circuit sera représenté par une boite :



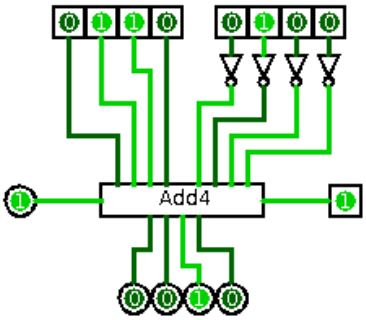
Ces circuits sont eux-mêmes utilisables pour réaliser des additionneurs de nombres plus long.

## 2 Soustraction – Rappel

Le codage en complément à 2 étant utilisée pour représenter l'opposé d'un entier, la soustraction est réalisée avec en additionnant le complément à deux de la valeur à soustraire. On utilise donc un additionneur modifié pour réaliser cette opération :

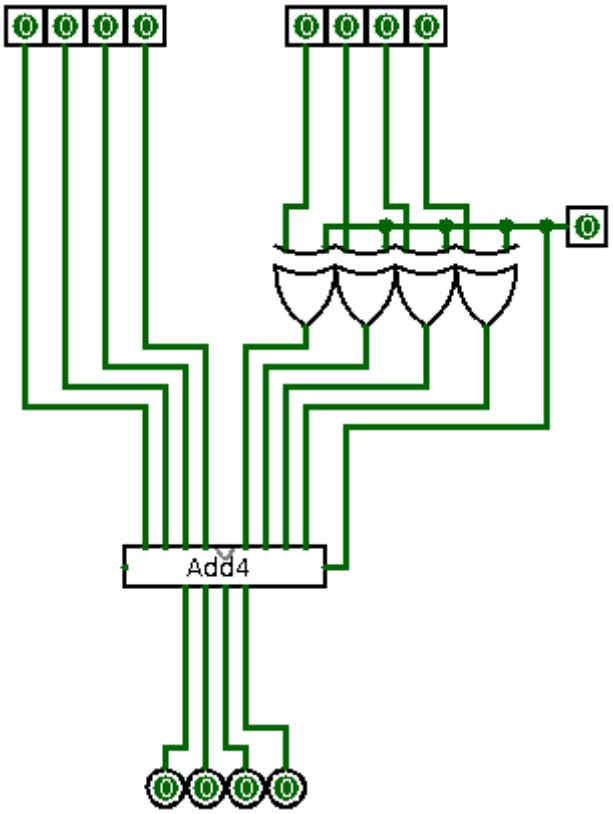
A – B = A + C2(B)
C2(B) : Complément à 2 de B

Pour obtenir le complément à 2 d'une valeur il suffit d'inverser tous ses bits et de lui ajouter 1. Le circuit est donc modifié ainsi :




Ce circuit illustre la soustraction : 6 – 4 = 2

Et en mélangeant les 2 circuits on peut fabriquer le circuit qui réalise soit l'addition soit la soustraction en fonction d'un entrée de choix d'opération :



## 3 Opérations logiques

Les opérations logiques que réalise un processeur sont simples, et peuvent même se limiter à des comparaisons avec une fonction qui prend 2 valeurs en paramètre et qui fonctionne ainsi :

- = : 1 si les valeurs sont égales, 0 sinon,
- < : 1 si la première valeur est inférieure à la seconde, 0 sinon,
- > : 1 si la première valeur est supérieure à la seconde, 0 sinon.

On peut construire ce circuit sur le modèle de l'additionneur, par exemple pour la fonction *Inferieur* <.
Soient deux nombres A et B de longueur N (nombre de chiffres). La comparaison de 2 chiffres se fait ainsi :

| $a_i$ | $b_i$ | $I_{i-1}$ | $I_i$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

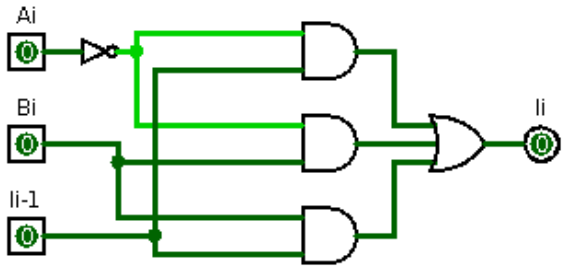
Cette table décrit la décision à prendre pour savoir si A est inférieur à B, en comparant les valeurs des chiffres de niveau i et connaissant les résultats des comparaisons des niveaux inférieurs à i :

- si $a_i < b_i$ alors on renvoie 1 au niveau supérieur : A < B pour l'instant,
- si $a_i > b_i$ alors on renvoie 0 au niveau supérieur : A > B pour l'instant,
- si $a_i = b_i$ alors on renvoie la valeur du niveau inférieur.

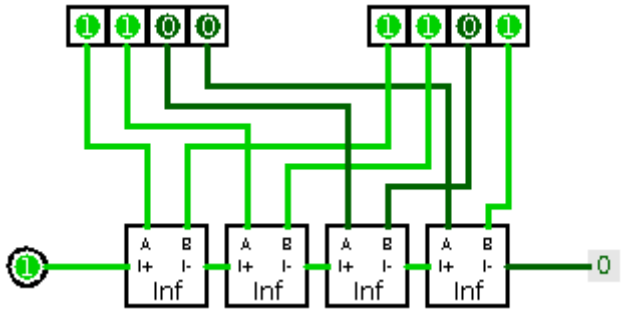
On obtient la fonction :

$$\overline{a_i}.b_i + \overline{a_i}.I_{i-1} + b_i.I_i$$

Et le circuit :




Le comparateur de nombres :




De même on peut construire les fonctions A > B, A = B et par la suite les fonctions A <= B, A >=B, A!= B.

La comparaison de nombres peut se faire aussi en utilisant un soustracteur et en regardant le signe de la différence.

## 4 État de l'ALU

Si on revient au schéma général de l'ALU, on a un circuit qui prend en entrée 2 valeurs et l'opération à effectuer, et qui fournit en sortie la valeur calculée et une valeur nommée état. En pratique c'est cet état qui est produit par le résultat des opérations de comparaison. Cette valeur est représentée par des bits nommés Flag qui sont généralement :

- Bit de zéro ou ZF pour Zero Flag : si le résultat de l'opération arithmétique a donné 0 comme résultat il sera à 1.
- Bit de débordement ou OF pour Overflow Flag : il sera mis à 1 si l'opération a généré un débordement, par exemple si l'addition de deux nombres positifs a donné un résultat négatif (ou le contraire). Il sera à 0 sinon.
- Bit de retenue ou CF pour Carry Flag : il sera mis à 1 si l'opération a généré une retenue, ce qui permet éventuellement d'augmenter (ou de conserver) la capacité de calcul de l'ALU, à condition de bien gérer cette information.
- Bit de signe ou SF : à 1 si le résultat de l'opération est négatif, à 0 s'il est positif.
- Bit de parité ou PF : à 1 si le résultat contient un nombre pair de 1.
- etc.

Ces états peuvent être réalisés par des circuits comme celui présenté plus haut, mais ils peuvent aussi être réalisés à partir des circuits arithmétiques :

a) ZF ou Zero Flag

Ce drapeau est réalisé en faisant un NOR sur les sorties du résultat arithmétique, ainsi cette sortie sera à 1 si et seulement si tous les bits du résultat sont à 1.

b) OF ou Overflow Flag

Le débordement peut se produire seulement lors d'une addition de 2 nombres de même signe, dans le cas ou le résultat est trop grand pour la représentation. Il sera à 1 si le signe du résultat est différent du signe des opérandes.

| SA | SB | SR | OF |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

$$OF = \overline{SA}.\overline{SB}.SR + SA.SB.\overline{SR}$$

c) CF ou Carry Flag

Il s'agit du bit de retenue de l'additionneur.

d) SF ou Sign Flag

Il s'agit du bit de signe du résultat.

e) Comparaison de 2 nombres

Si on réalise la soustraction de 2 nombres positifs :

- il suffit de regarder ZF pour savoir si les 2 nombres sont égaux
- il suffit de regarder SF pour savoir lequel est le plus grand.

f) PF ou Parity Flag

Ce drapeau n'est pas forcément présent, mais si on souhaite l'avoir dans son processeur, il suffit de réaliser un NON XOR entre les sorties du résultat pour l'obtenir.

On verra plus tard comment utiliser dans les instructions machines du processeur ces différents drapeaux.

## 5 Autres opérations

Une ALU est capable de réaliser de nombreuses autres opérations plus ou moins simples. Classiquement les opérations simples portant sur des entiers seront réalisés par l'ALU principale :

- opérations logiques :
  - NON sur un opérande,
  - OU, ET, XOR... entre les opérandes,
  - complément à 1 d'un opérande (inversion 1 ↔ 0),
- opérations arithmétiques simples :
  - changement de signe d'un opérande (attention à la signification),
  - complément à 2 d'un opérande,
  - multiplication par 2, il suffit de décaler les valeurs à gauche,
  - division par 2, il suffit de décaler les valeurs à droite,
  - incrémentation (plus 1) en utilisant un seul opérande,
  - multiplication, division, modulo (ces opérations peuvent être confiée à la FPU cf. plus loin),
  - etc.

On utilisera une ALU particulière pour les opérations sur les nombres en virgules flottantes, appelée FPU (Floating Point Unit) qui peut prendre en charge des opérations complexes :

- multiplication, division, modulo, inverse d'un opérande,
- racine carrée,
- logarithmes,
- fonctions trigonométriques (sinus, cosinus...),
- etc.

Il existe aussi des processeurs de calcul vectoriel capable d'effectuer une même opération sur plusieurs données à la fois...

Évidemment, dans un ordinateur on peut trouver plusieurs ALU (et FPU) pour augmenter la capacité de calcul de la machine.